An Effective Local Search Algorithm for the Multidepot Cumulative Capacitated Vehicle Routing Problem

Xinyu Wang, Tsan-Ming Choi, Zhiying Li, and Shuai Shao

***Abstract*—The cumulative capacitated vehicle routing problem (CCVRP) focuses on minimizing the sum of arrival times at the customers. An important application of the CCVRP arises in the procurement of humanitarian aid when natural disasters occur. In this article, the multidepot CCVRP (MDCCVRP) is investigated, and its mathematical model is developed. Moreover, an effective perturb-based local search (PLS) algorithm is proposed to solve the problem. The proposed PLS algorithm starts from having a feasible solution. Regarding the PLS, the perturbing operators help to explore the searching space, while the six local search operators help to exploit the best solution within each searching basin. To test the performance, the proposed PLS is applied to a set of standard benchmark instances and compared with the recently published algorithms. Extensive computational studies reveal that the proposed PLS algorithm is able to attain better solutions with less computational time for most testing instances, and thus competes very favorably with the previously proposed approaches. A stability analysis is also presented to shed light on the robust behavior of the proposed algorithm. The results of the two-sided Wilcoxon rank sum tests clearly show that the algorithms (our PLS and other published methods) have performed with statistically significant differences.**

***Index Terms*—Humanitarian aid, local search, multidepot cumulative capacitated vehicle routing problem (MDCCVRP), perturbing operator.**

I. Introduction

THE CAPACITATED vehicle routing problem (CVRP) and its variants play an important role within the area of transportation systems optimization [1]–[3]. The cumulative capacitated vehicle routing problem (CCVRP), originally proposed by Ngueveu *et al.* [4], aims at minimizing the sum of arrival times of vehicles at the customers. The CCVRP is a relatively new variant of the capacitated vehicle routing problem (CVRP). When natural disasters strike, it is essential that aid arrives quickly in order to reduce casualties and life losses [5]–[7]. For fast response, the CCVRP can meet the requirement of providing a humanitarian supply [8], [9].

The multidepot CVRP (MDCVRP) is another variant of the CVRP, in which customers can be served by vehicles from different depots [10]. There are a lot of real-life transportation problems, in which vehicles can depart from several depots [11]. When the disaster strikes, the relief items are always transported from several temporary warehouses to the affected areas. Therefore, it is meaningful to investigate the multidepot CCVRP (MDCCVRP). To the best of our knowledge, in the literature, only [12] has explored the MDCCVRP.

The CCVRP is proven to be an NP-hard problem [4], and the MDCCVRP is more complex than the CCVRP [12]. Lalla-Ruiz and Voß [12] investigated the MDCCVRP and proposed a POPMUSIC approach for solving the problem. The underlying idea behind the POPMUSIC method is to address large problems by first decomposing a problem into parts, and subsequently, bundling some parts to build subproblems. The subproblems are solved with an exact approach. Because the practical large-scale MDCCVRP instances are difficult to be tackled efficiently by exact solution approaches, a simple and efficient heuristic is proposed for solving the MDCCVRP. The developed method should preferably be fast, and able to handle large problems.

The local search method for the CVRP was first introduced by Shaw [13]. The local search method is simple and flexible, and it can yield high-quality solutions very quickly. A lot of local search-based heuristics are proposed to solve the CVRP and its variants [14]–[16]. Traditional local search method has the problem of search becoming trapped in the local minima, a widespread use of minima escaping scheme is introducing perturbing process to lead the search out of those traps. Motivated from that a novel perturb-based local search (PLS) algorithm is proposed in this article for solving the MDCCVRP.

The main contributions of this article are: 1) establishing the practical and useful mathematical model for the MDCCVRP; 2) developing a simple and efficient algorithm PLS for solving the MDCCVRP; 3) simplifying the solution representation and optimization operators by introducing the virtual subtours;

Manuscript received May 27, 2019; accepted August 16, 2019. Date of publication September 19, 2019; date of current version November 18, 2020. This work was supported in part by the general project of Humanities and Social Sciences Research of the Ministry of Education under Grant 18YJC630185, in part by the National Natural Science Foundation of China under Grant 61402086 and Grant 71602021, in part by the International Cooperation and Exchange of the National Natural Science Foundation of China under Grant 71110107024, and in part by the Key Project of the Natural Science Foundation of China under Grant 71831003. This article was recommended by Associate Editor M. K. Tiwari. *(Corresponding author: Tsan-Ming Choi.)*

X. Wang and S. Shao are with the Department of Management Science and Engineering, Dongbei University of Finance and Economics, Dalian 116025, China (e-mail: distribute2008@163.com; shaoshuai3112@163.com).

T.-M. Choi is with the Institute of Textiles and Clothing, Hong Kong Polytechnic University, Hong Kong (e-mail: jason.choi@polyu.edu.hk).

Z. Li is with the School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing, China (e-mail: 18742054997@163.com).

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TSMC.2019.2938298

2168-2216 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

and 4) designing a smart perturbing scheme to improve the performance of the proposed algorithm.

The outline of this article is as follows. Section II provides the related literature review. The mathematical formulation for the MDCCVRP is introduced in Section III. The proposed PLS algorithm is presented in Section IV. In Section V, the computational results and analysis are presented. Finally, the conclusions and future research directions are discussed in Section VI.

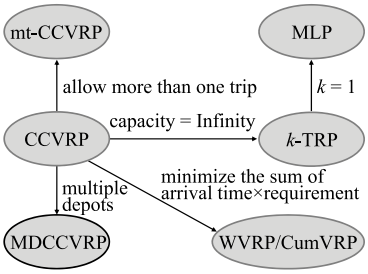


Fig. 1. Relationship between CCVRP and the other five related problems.

II. Literature Review

In this review, we mainly focus on the following streams of literature that are close to this article: 1) CCVRP; 2) multitrip CCVRP (mt-CCVRP); 3) k-traveling repairman problem (k-TRP); 4) minimum latency problem (MLP); 5) weighted vehicle routing problem (WVRP); and 6) MDCCVRP.

CCVRP: The CCVRP aims at minimizing the sum of arrival times of vehicles at customers under the constraints of vehicle capacity. Ngueveu *et al.* [4] proposed the mathematical formulation, and presented the first upper and lower bounding procedures for the CCVRP. Lysgaard and Wøhlk [8] developed an exact algorithm (branch-and-cut-and-price) to deal with the problem. Moreover, the CCVRP has been recently widely studied from the heuristic point of view. The studies include memetic algorithms (MAs) [4], adaptive large neighborhood search (ALNS) [17], skewed variable neighborhood search (SVNS) [18], two-phase heuristic (TPH) [19], brain storm optimization algorithm [20], and iterated greedy algorithm (IGA) [9]. Liu and Jiang [21] proposed a hybrid method based on large neighborhood search and genetic algorithm for solving the CCVRP with time windows.

mt-CCVRP: The mt-CCVRP is a nontrivial extension of the CCVRP, in which each vehicle may perform multiple trips. Rivera *et al.* [22] proposed a mixed integer linear programming (MILP) for the mt-CCVRP. The MILP could solve small instances with up to 15 customers. Moreover, Rivera *et al.* [22] developed a multistart iterated local search algorithm which produced good solutions on the large instances with more than 200 customers. The CCVRP with a single vehicle can be viewed as a special case of the mt-CCVRP, where only one vehicle can be used and the vehicle can perform multiple trips [23]. Based on a multilevel network, Martinez-Salazar *et al.* [24] proposed two mixed integer formulations to solve the mt-CCVRP with only one single vehicle. The methods could solve to optimality instances with up to 30 customers. Also, Martinez-Salazar *et al.* [24] developed a heuristic algorithm to produce high-quality solutions within a short computational time. The algorithm designed a composite method which uses intraroute and interroute "*move*" operations in an iterative way.

k-TRP: The k-TRP is the uncapacitated version of the CCVRP, where k is the number of vehicles available. Fakcharoenphol *et al.* [25] proposed an 8.497α-approximation algorithm to solve the k-TRP, where α denotes the approximation factor for finding the least cost rooted tree spanning problem. Nucamendi-Guillen *et al.* [26] built a novel mixed integer formulation for the k-TRP, which could find the optimal solution for problems with up to 80 customers. Moreover, Nucamendi-Guillen *et al.* [26] proposed an IGA which could obtain the high-quality solutions for large-size instances within a short computational time.

MLP: The MLP is a variant of the well-known traveling salesman problem (TSP). The objective of the MLP is to minimize the sum of waiting time for each customer [27]. The MLP can be viewed as a special case of the k-TRP where k is limited to 1. To solve the MLP, Bulhoes *et al.* [28] proposed a branch-and-price method over a set partitioning formulation. The algorithm strongly relies on *ng*-paths, where the columns represent *ng*-paths. The column generation bounds computed on each node are derived from the dynamically defined *ng*-sets.

WVRP: Kara *et al.* [29] considered a variant of the CVRP (referred as CumVRP), where the objective function is minimizing the sum of arrival times multiplied by the demand of the customer. The authors of [29] proposed a flow-based formulation for the CumVRP in the delivery and the collection cases. Tang *et al.* [30] proposed the WVRP, where the objective function is minimizing the total cost generated by both the traveling distance and the loading weight. The CumVRP and the WVRP are equivalent. The authors of [30] developed a BEAM-MMAS algorithm that combines the max–min ant system with a beam search to solve the WVRP. To solve the multidepot version of the WVRP, Wang *et al.* [31] proposed an ACOMR algorithm which introduces a smart design of the ants' tabus to simplify the solution constructing process. The MDCVRP with loading cost (MDCVRPLC) is an extension of the WVRP, and Kuo *et al.* proposed a variable neighborhood search (VNS) to determine the solution.

MDCCVRP: The MDCCVRP hepls to capture the real-world scenarios where the service level depends on the customer waiting time and the delivering vehicles can depart from different depots. Lalla-Ruiz and Voß [12] formalized the MDCCVRP by means of a mathematical formulation and proposed a POPMUSIC approach for solving the problem. The POPMUSIC method could solve small instances efficiently.

In the aftermath of a disaster, the relief items are transported to several temporary warehouses. Besides, in vital goods transportation, goods may be stored in several warehouses. Therefore, multiple depots should be considered when designing the optimal vehicle routes in practice. On the other hand, reaching demand points to provide fast and fair service is a priority. The MDCCVRP is hence critical and important. Fig. 1 shows the relationship between the CCVRP and the other five problems.

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

TABLE I
LIST OF NOTATION USED IN MDCCVRP

Notation	Definition
D	Set of depots
C	Set of customers
R	Set of vehicles
N_i	Max number of vehicles at depot i, $i \in D$
q_j	Requirement at customer j, $j \in C$
Q_k	Capacity of vehicle k, $k \in R$
t_{ij}	Travel time from i to j, $i, j \in D \cup C$
y_i^k	Visiting time of vehicle k at i, $k \in R$, $i \in D \cup C$
x_{ij}^k	Decision variable, $k \in R$, $i, j \in D \cup C$
G	Large positive constant

III. MATHEMATICAL FORMULATION OF THE MDCCVRP

The MDCCVRP can be defined on a directed graph $G = (V, E)$. Let $C = \{c_1, c_2, \ldots, c_n\}$ and $D = \{d_1, d_2, \ldots, d_m\}$, respectively, be the set of customers and the set of depots. Then, V ($V = D \cup C$) is the set of all vertices in the graph G. Each customer must be visited exactly once. Table I describes the notation for the definition of the MDCCVRP. The variable y_i^k denotes the arrival time of vehicle k at node i. If vehicle k does not serve i, y_i^k is equal to 0. If i is a certain depot, y_i^k is set to 0. The decision variable x_{ij}^k is equal to 1 if vehicle k passes the path from i to j, otherwise 0. The MDCCVRP aims to minimize the sum of arrival times of vehicles at the customers. The mathematical formulation of the MDCCVRP can be stated as follows:

$$\text{minimize } f(x) = \sum_{k \in R} \sum_{i \in C} y_i^k \tag{1}$$

$$\text{s.t.} \quad \sum_{j \in V} x_{ji}^k = \sum_{j \in V} x_{ij}^k \quad \forall i \in C \ \forall k \in R \tag{2}$$

$$\sum_{k \in R} \sum_{i \in V} x_{ij}^k = 1 \quad \forall j \in C \tag{3}$$

$$\sum_{i \in D} \sum_{j \in C} x_{ij}^k = \sum_{i \in D} \sum_{j \in C} x_{ji}^k \quad \forall k \in R \tag{4}$$

$$\sum_{i \in D} \sum_{j \in C} x_{ij}^k \leq 1 \quad \forall k \in R \tag{5}$$

$$\sum_{i \in V} \sum_{j \in C} x_{ij}^k \cdot q_j \leq Q_k \quad \forall k \in R \tag{6}$$

$$\sum_{k \in R} \sum_{j \in C} x_{ij}^k \leq N_i \quad \forall i \in D \tag{7}$$

$$y_i^k + t_{ij} - (1 - x_{ij}^k) \cdot G \leq y_j^k, \quad \forall i \in V \ \forall j \in C \ \forall k \in R \tag{8}$$

$$y_i^k = 0 \ \forall k \in R \ \forall i \in D \tag{9}$$

$$y_i^k \geq 0 \quad \forall i \in C \ \forall k \in R \tag{10}$$

$$x_{ij}^k \in \{0, 1\} \quad \forall i, j \in V, i \neq j \ \forall k \in R. \tag{11}$$

The objective function (1) minimizes the sum of arrival times at the customers. Constraint (2) ensure that a vehicle arriving at customer i ($i \in C$) must leave it. Constraint (3) ensure that each customer is visited by exactly one vehicle. Constraints (4) and (5) ensure that the depot is at the beginning and at the end of each route, and each vehicle can be scheduled for at most one trip. Constraint (6) ensure that the total demand does not exceed the vehicle's capacity. Constraint (7) ensure that vehicles used should not exceed the number of available vehicles at each depot. Constraint (8) ensure that if customer j is visited immediately after customer i by vehicle k, then, y_j^k must be no less than $y_i^k + t_{ij}$. Note that G is a sufficiently large positive constant, and the constraints help eliminate subtours. Finally, constraints (9)–(11) impose restrictions on the decision variables, which follow their physical meaning.

Algorithm 1 Main Procedure of the PLS Algorithm

Input: An MDCCVRP instance to be solved
Output: The globally best solution *gbest*

```
procedure
    Initialize s // Described in SubSection IV.B
    Initialize the two parameters λ_i and λ_o
    Sign = false, w_i = w_o = 0
    while Sign == false do
        gbest = local_search(s) //Described in SubSection IV.C
        if gbest is updated during this round then
            w_i = 0, w_o = 0
        end if
        if w_i < λ_i then
            Perturb s with the random perturbation operator
            w_i = w_i + 1
        else if w_o < λ_o then
            Perturb s with the remove-insert perturbation operator
            w_i = 0
            w_o = w_o + 1
        else
            Sign = true
        end if
    end while
end procedure
```

IV. PROPOSED ALGORITHM FOR MDCCVRP

The MDCCVRP is NP-hard. The exact algorithm always produces optimal solutions, while it is very time consuming. It is hence critical to give a relatively good "satisficing" solution within a short time, rather than the "best" solution after a long "intolerable" time [32]–[35]. The developed method should preferably be fast, and able to handle large problems. Thus, it seems fair to turn to heuristic methods.

Local search is a simple, powerful, and generally applicable stochastic method [36]. Local search has provided good solutions for other VRP variants (see [14]–[16], [22]). Starting from an initial solution, the local search method usually leads to a local minimum. In order to emerge from the corresponding valley, a perturbing or a restarting process will be introduced in the local search [14], [15], [36]. In this article, a PLS heuristic is developed to solve the MDCCVRP. The main procedure of the proposed PLS algorithm is given in Algorithm 1. In the following, the solution representation and initialization as well as the local search operators and perturbation operators are introduced in detail.

A. Solution Representation

The symbols used in this section are in accordance with those shown in Table I. A solution is represented by g subtours, where $g = \sum_{i \in D} N_i$. Each subtour starts from a depot and returns to the departing depot. When a subtour includes customer nodes, the subtour is an actual one. Otherwise, a subtour consisting of only two nodes (starting depot and ending

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

$D = \{d_1, d_2\}$

$C = \{c_1, c_2, ..., c_8\}$

$N_1 = N_2 = 2$

$|R| = 3$

subtour$_1$:	d_1—c_1—c_2—c_7—d_1
subtour$_2$:	d_2—c_3—c_4—d_2
subtour$_3$:	d_2—c_5—c_6—c_8—d_2
subtour$_4$:	d_1—d_1

Fig. 2. Example of solution representation.

depot) is a virtual one. Given a feasible solution, its actual subtours should be no more than $|R|$, and each customer appears in only one subtour, and the total requirement of each subtour should be no more than the vehicle capacity. Fig. 2 shows an example of solution representation. The solution consists of three actual subtours and one virtual subtour.

B. Solution Initialization

A feasible solution is initialized in a constructive manner. First, g ($g = \sum_{i \in D} N_i$) virtual subtours are constructed, where there are N_i ($i \in D$) virtual subtours starting from depot i. The g virtual subtours can be viewed as a partial solution. Then, the customers are selected one by one, and each selected customer will be inserted into the proper subtour. Two important decisions should be made: 1) selecting the next customer from the remaining customers and 2) determining the proper subtour for the selected customer. A simple regret-insertion-based heuristic [17] is used to make the two decisions, which provides a look-ahead mechanism during the construction process.

Let Δ_i^j be the inserting cost incurred by inserting customer i at the end of subtour j. For customer i, there are g inserting costs, and the smallest (and the third smallest) insertion cost is labeled as C_i^1 (and C_i^3). The regret value (RV) of customer i is defined as $(C_i^3 - C_i^1)$. The heuristic always selects the customer with the largest RV. Then, the selected customer will be inserted into the subtour with the smallest inserting cost. Once the partial solution is updated, the RV for each remaining customer will be recomputed. The inserting process is repeated until all customers are inserted. To guarantee at most $|R|$ vehicles are used, at most $|R|$ actual subtours are allowed in the construction process. Once there are $|R|$ actual subtours in the partial solution, the RVs are calculated based on the actual subtours instead of all the g subtours.

Pseudocode for the construction process of the initial solution is shown in Algorithm 2. The symbols used in Algorithm 2 follow those shown in Table I.

C. Solution Improvement

A local search method iteratively improves a solution by exploring its neighborhood. In the implementation of a local search algorithm, four critical components are: 1) initial feasible solution; 2) the *move* operators for generating neighborhood solutions; 3) acceptance criterion; and 4) perturbation scheme. In the previous sections, the solution representation and initialization were discussed. Given a feasible solution represented by a set of subtours $s = \{T_1, T_2, \ldots, T_g\}$, where

Algorithm 2 Solution Initialization Algorithm

Input: An MDCCVRP instance to be solved
Output: An initial solution

```
procedure
    Create g virtual subtours
    /* AT denotes the number of actual subtours, and RC denotes the set of remaining
    customers, respectively. */
    AT = 0, RC = C
    while RC ≠ ∅ do
        if AT < |R| then t = g
        else    t = |R|
        end if
        for each x in RC do
            ComputeRegretValue(x, t)
        end for
        Label the customer with the largest RV as u
        For u, label the subtour with the smallest insertion cost as ξ
        if ξ is a virtual subtour then
            AT = AT + 1
        end if
        Insert u at the end of subtour ξ, and remove u from RC
    end while
end procedure
```

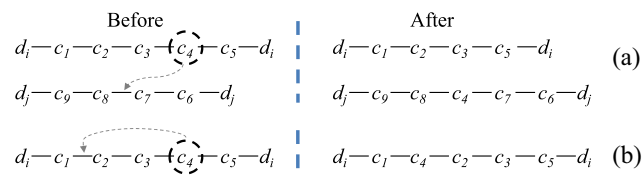


Fig. 3. Example of the relocation *move* (a) intersubtour *move* and (b) intrasubtour *move*.

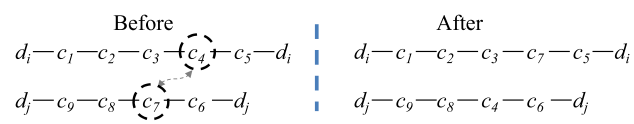


Fig. 4. Example of the 2-exchange *move*.

subtour T_i specifies the starting depot and the customers serviced by vehicle i. If subtour T_i contains only the depot node, the subtour is a virtual one.

The *move* operator helps generate the neighboring solutions for a given solution. In our proposed algorithm, six *move* operators are used to search for the neighborhood. A combination of multiple operators can make the space search more extensive [36]. Details about the six *move* operators are introduced as follows.

1) Relocation Move: The relocate *move* operator deletes a customer from a subtour, and then inserts the deleted customer in between two other customers who occupy successive positions. The relocate *move* operator may occur within one subtour or between two different subtours. An example of the relocation *move* is shown in Fig. 3.

2) 2-Exchange Move: The 2-exchange *move* operator swaps two customers or two depots in different subtours. A 2-exchange *move* operation can be implemented by a combination of two intersubtour relocation *move* operations. An example of the 2-exchange *move* is shown in Fig. 4.

3) 2-Opt Move: The 2-opt *move* operator tries to improve the solution by replacing two arcs with two new arcs. Suppose that the arcs (c_i, c_j) and (c_s, c_r) in the original solution are selected. If the two selected arcs belong to the same subtour, the two arcs are replaced by two new arcs (c_i, c_s) and (c_j, c_r),

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

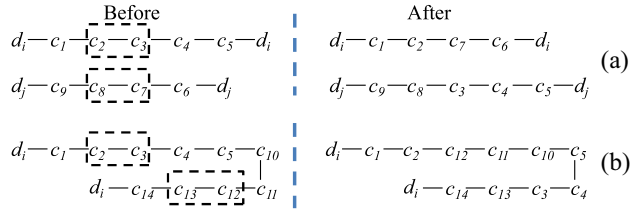


Fig. 5. Example of the 2-opt *move* (a) intersubtour *move* and (b) intrasubtour *move*.

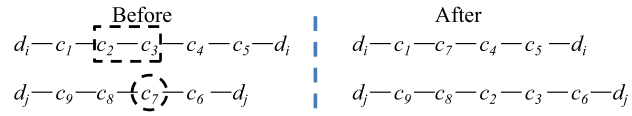


Fig. 6. Example of the arc-node exchange *move*.

thus reversing the direction of customers between c_j and c_s. If the two selected arcs belong to different subtours, the two arcs are replaced by two new arcs (c_i, c_r) and (c_s, c_j). Fig. 5(a) shows an example of the 2-Opt operator applied between two different subtours. The two arcs (c_2, c_3) and (c_8, c_7) in the original solution are replaced by two new arcs (c_2, c_7) and arc (c_8, c_3) in the new solution. Fig. 5(b) shows an example of 2-Opt operator applied within one subtour. The two arcs (c_2, c_3) and (c_{12}, c_{13}) in the original solution are replaced by two new arcs (c_2, c_{12}) and arc (c_3, c_{13}), thus reversing the direction of customers between c_3 and c_{12}.

4) Arc-Node Exchange Move: The arc-node exchange *move* randomly selects a customer and an arc, and then exchanges their positions. Like the 2-exchange *move*, the operator involves two different subtours. An example of the arc-node exchange *move* is shown in Fig. 6. As shown in the figure, the arc (c_2, c_3) and the node c_7 are selected. The two arcs (c_1, c_2) and (c_3, c_4) are replaced by arcs (c_1, c_7) and (c_7, c_4). The two arcs (c_8, c_7) and (c_7, c_6) are replaced by arcs (c_8, c_2) and (c_3, c_6).

5) Or-Opt Move: The or-opt operator introduced in [36] only considers the situation of intrasubtour *move*. The basic idea is to relocate a chain of consecutive customers. Like the relocation *move*, the new position of the transferred chain may be in the same subtour, or in a different one. This is performed by replacing three arcs with three new arcs, preserving the orientation of each subtour. An example of the or-opt *move* is shown in Fig. 7. For example, in Fig. 7(a), the arcs (c_1, c_2), (c_4, c_5), and (c_8, c_7) in the original solution are replaced by the three new arcs (c_1, c_5), (c_8, c_2), and (c_4, c_7) in the destination solution. In Fig. 7(b), the arcs (c_1, c_2), (c_4, c_5), and (c_{11}, c_{12}) in the original solution are replaced by the three new arcs (c_1, c_5), (c_{11}, c_2), and (c_4, c_{12}) in the destination solution.

6) Cross-Exchange Move: In the cross-exchange *move*, the basic idea is to swap two segments from different subtours. The operator removes two arcs (c_i, c_j) and (c_r, c_t) from one subtour, and similarly removes two arcs (c_a, c_b) and (c_u, c_v) from another subtour. Then, the segments $c_j - \cdots - c_r$ and $c_b - \cdots - c_u$, which may contain an arbitrary number of customers, are swapped by introducing the new arcs (c_i, c_b), (c_u, c_t), (c_a, c_j), and (c_r, c_v). An example of the cross-exchange

Algorithm 3 *local_search* Method.

Input: A solution s to be improved
Output: A local optima

```
procedure
    Let Γ be the set of the six move operators
    Flag = true
    while Flag = true do
        Flag = false
        for each γ in Γ do
            Improve(s, γ)
            if s is improved with γ then Flag = true
            end if
        end for
    end while
end procedure
```

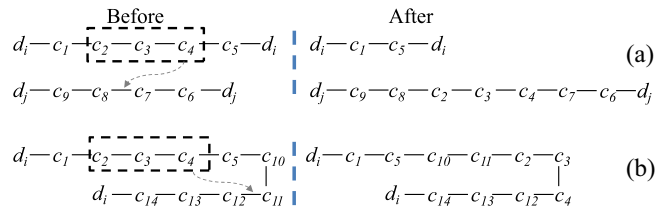


Fig. 7. Example of the or-opt *move* (a) intersubtour *move* and (b) intrasubtour *move*.

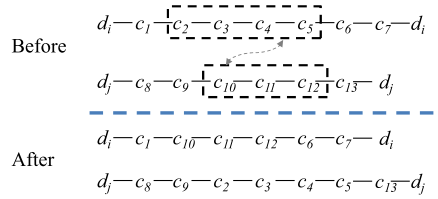


Fig. 8. Example of the cross-exchange *move*.

move is shown in Fig. 8. The four arcs (c_1, c_2) and (c_5, c_6), (c_9, c_{10}) and (c_{12}, c_{13}) in the original solution are replaced by the new four arcs (c_1, c_{10}) and (c_{12}, c_6), (c_9, c_2) and (c_5, c_{13}). In other words, the segment c_2–c_3–c_4–c_5 and the segment c_{10}–c_{11}–c_{12} are exchanged.

The six *move* operators are invoked one by one. Each *move* operator helps lead the solution to a local optima. Once the solution cannot be improved with the operator, the next *move* operator will be invoked. In each *move* operator, "first-accept" is always taken as the acceptance criteria. Local search will end when no more improvement can be achieved. Procedure of the *local_search* method used in Algorithm 1 is shown in Algorithm 3.

D. Perturbation Operators

A good heuristic exploits the best solution by combining local search with perturbations. The perturbation operators are typically introduced for diversification purposes. Two simple perturbations are used: random perturbation and remove-insert perturbation.

The random perturbation is based on two operators (2-Exchange and 2-Opt). This perturbation randomly accepts a feasible solution from its corresponding neighborhood. The random perturbation is applied H times on the local optima before resubjecting it to the local search improvement. Similar to [19], H is randomly chosen from [2, min($|R|$/2, 10)], where $|R|$ refers to the number of vehicles. Our random perturbation

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

procedure adopts these two operators since they are simple and work well for the MDCCVRP.

In the remove-insert perturbation, the basic idea is to remove customers with a high cost in the current solution s, and to insert them in better positions. Thus, this perturbation is based on the relocation *move* operator. Observe that in the literature, Li *et al.* [37] proposed an efficient perturbation operator successfully used for the CVRP. Motivated by [37], the remove-insert perturbation for MDCCVRP is proposed by adjusting the computation of cost difference. For each customer i in solution s, define $\varphi(i, s) = q_i/\delta(i, s)$, where q_i represents the demand at customer i and $\delta(i, s)$ represents the difference between the solution cost of s and the solution cost without customer i in s. Sort the φ values of all the customers in ascending order and select the first r customers, where $r = \min(20, |C|/10)$($|C|$ denotes the number of customers). The parameter r is set to be the same value as that introduced in [37]. Then, the perturbation operator tries to reinsert these r customers, one by one, into a new location of a subtour using least-cost insertion while maintaining feasibility.

V. Computational Results and Analysis

The proposed PLS algorithm is compared with the general GRASP construction algorithm [38], VNS algorithm [39], and POPMUSIC algorithm [12]. These three algorithms are selected for comparisons because: 1) the GRASP construction algorithm is often adopted to quickly produce good solutions; 2) the VNS algorithm was proposed to solve the MDCCVRP-related problem (i.e., MDVRPLC) in the past and has been proven to be effective [39]; and 3) the POPMUSIC algorithm is the state-of-the-art approach for solving the MDCCVRP. The computer codes for VNS were kindly provided by the corresponding author (Kuo) of [39]. The computer codes for VNS was modified so as to solve the MDCCVRP. The proposed PLS was coded in *C*++. All heuristics were tested on a PC equipped with 4G MB of RAM and Intel $i5 - 4210M$ 2.60-GHz dual-core processor. To ensure fair comparisons, all heuristics were run 30 times for each instance. The travel times are equal to the Euclidean distance (accuracy: two decimal places).

In our PLS algorithm, the two parameters λ_i and λ_o are set to be 3 and 20, respectively. In our preliminary test, it is observed that the PLS performs very well by using the parameter settings. Though it is possible to find a better parameter configuration, the computational experiments reported in this section show that the adopted parameter settings perform globally well on the testing instances. As for the parameter settings of VNS and POPMUSIC, the configuration given in [12] and [39] was adopted. GRASP is a well-known constructive metaheuristic, which starts from an empty solution and builds a complete solution by adding one element at a time [38]. No iterative improvement will be applied to the final solution. Hence, GRASP works much faster than the other methods.

A. Data Sets

The experiments were conducted on the 33 MDCVRP testing instances, all of which are publicly available from the website: http://www.bernabe.dorronsoro.es/vrp/. The instances are divided into S_1 (p01–p23) and S_2 (pr01–pr10), and have various sizes ranging from 48 to 360 customers. Detailed data and numbers on the standard MDCVRP instances are shown in columns 1 to 4 of Table II. The experiments were also conducted on the 18 instances S_3 (lr01–lr18) proposed by Lalla-Ruiz and Voß [12], all of which are publicly available from the website: https://github.com/elalla/MDCCVRP. The instances lr01∼lr03, instances lr04∼lr06, instances lr07∼lr12, and instances lr13∼lr18 consider 10, 25, 50, and 100 customers, respectively.

TABLE II
Results of Best-Known Solution for the MDCVRP and the Obtained Best Solution for the MDCCVRP

No.	$\|D\|$	$\|C\|$	$\|R\|$	F(MDCVRP)	F(MDCCVRP)	GAP_1
p01	4	50	11	1444.33	1055.35	-26.93%
p02	4	50	5	2493.17	2043.13	-18.05%
p03	5	75	11	2252.61	1758.70	-21.93%
p04	2	100	15	3511.11	2618.44	-25.42%
p05	2	100	8	4957.48	3766.20	-24.03%
p06	3	100	16	2968.14	2160.93	-27.20%
p07	4	100	16	2949.11	2142.11	-27.36%
p08	2	249	25	22970.69	17393.46	-24.28%
p09	3	249	26	20162.76	15041.69	-25.40%
p10	4	249	26	18884.53	14265.77	-24.46%
p11	5	249	26	17924.75	14381.43	-19.77%
p12	2	80	8	6494.05	5494.36	-15.39%
p13	2	80	9	6494.05	4914.66	-24.32%
p14	2	80	10	6494.05	4510.12	-30.55%
p15	4	160	16	14454.52	10662.27	-26.24%
p16	4	160	17	14454.52	10086.50	-30.22%
p17	4	160	18	14454.52	9538.69	-34.01%
p18	6	240	24	22191.46	15912.27	-28.30%
p19	6	240	25	22191.46	15255.02	-31.26%
p20	6	240	26	22191.46	14709.23	-33.72%
p21	9	360	34	33770.59	25770.35	-23.69%
p22	9	360	35	33770.59	24451.01	-27.60%
p23	9	360	36	33770.59	23656.13	-29.95%
AVG						-26.09%
pr01	4	48	4	5310.62	3748.11	-29.42%
pr02	4	96	8	9980.64	4973.36	-50.17%
pr03	4	144	11	12715.31	8357.54	-34.27%
pr04	4	192	14	14606.84	9273.89	-36.51%
pr05	4	240	19	15404.17	10075.01	-34.60%
pr06	4	288	23	18165.77	11053.59	-39.15%
pr07	6	72	6	7056.77	4877.86	-30.88%
pr08	6	144	12	10856.05	7141.88	-34.21%
pr09	6	216	17	13201.52	9235.29	-30.04%
pr10	6	288	24	18303.89	11643.69	-36.39%
AVG						-35.56%

TABLE III
Results Obtained From Different Methods for Emergency Transportation of Medical Supplies in the Case Study

Aspect	CPLEX	POPMUSIC	PLS
Cost	151006.00	150642.00	148794.00
Time(s)	7200.00	110.16	1.13

B. Comparisons Between the MDCVRP and the MDCCVRP

As discussed in [19], the best solution for the CVRP model is not the best one for the CCVRP model. Similarly, the best

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
RESULTS OBTAINED FROM DIFFERENT HEURISTIC ALGORITHMS FOR THE INSTANCE SET S_3 PROPOSED IN [12]

No.	POPMUSIC			VNS(200000)			VNS(500000)			PLS		
	Best	Avg	Time(s)	Best	Avg	Time(s)	Best	Avg	Time(s)	Best	Avg	Time(s)
lr01	**545.69**	556.64	3.41	**545.69**	547.66	2.23	**545.69**	545.69	5.52	**545.69**	554.97	0.15
lr02	**832.69**	841.90	2.23	**832.69**	833.22	2.24	**832.69**	832.69	5.71	**832.69**	847.80	0.14
lr03	**832.78**	834.74	3.72	**832.78**	832.78	2.27	**832.78**	832.78	5.75	**832.78**	841.47	0.14
lr04	2,180.54	2,420.91	1703.00	**2,082.28**	2,109.98	6.15	**2,082.28**	2,105.67	15.20	**2,082.28**	2,099.58	0.63
lr05	2,054.77	2,188.40	2401.00	**1,827.41**	1,884.44	5.99	**1,827.41**	1,865.29	14.89	**1,827.41**	1,870.90	0.65
lr06	1,953.68	2,075.25	2401.00	**1,786.95**	1,823.68	6.36	**1,786.95**	1,804.74	15.54	**1,786.95**	1,808.86	0.65
lr07	15,791.78	21,666.61	1201.00	5,426.24	5,455.06	18.16	**5,424.57**	5,436.40	44.30	**5,424.57**	5,440.37	2.72
lr08	18,312.43	18,507.63	1197.00	3,740.79	3,785.95	19.23	3,749.98	3,768.65	47.20	**3,737.38**	3,759.67	2.49
lr09	15,190.85	19,329.04	1201.00	3,817.24	3,864.90	18.85	3,805.51	3,843.56	46.20	**3,802.88**	3,811.65	2.68
lr10	5,202.64	10,991.71	2402.00	2,869.43	2,909.26	18.29	2,874.79	2,895.53	46.29	**2,868.39**	2,883.50	6.10
lr11	9,780.76	10,402.21	2102.00	2,988.06	3,023.27	18.84	**2,983.66**	3,013.68	47.37	2,987.75	3,008.88	6.37
lr12	8,485.06	9,304.31	2103.00	3,096.09	3,148.57	19.00	**3,090.38**	3,117.99	46.92	3,095.52	3,112.60	6.15
lr13	45,659.96	45,659.96	1205.00	8,387.08	8,521.41	34.63	8,379.59	8,461.82	86.10	**8,293.42**	8,331.27	6.11
lr14	46,682.08	46,682.08	1205.00	7,367.50	7,534.65	34.80	7,323.86	7,414.72	85.59	**7,273.03**	7,353.68	5.50
lr15	38,390.17	38,390.17	1205.00	8,717.87	8,792.87	34.92	8,643.29	8,736.67	85.49	**8,626.13**	8,644.64	7.27
lr16	23,005.64	27,017.76	1807.00	5,369.31	5,598.49	35.51	5,352.01	5,545.37	88.03	**5,306.50**	5,483.36	7.04
lr17	32,661.61	38,297.44	1807.00	6,170.57	6,399.96	34.07	6,144.23	6,328.73	84.08	**6,141.07**	6,265.57	7.55
lr18	29,813.45	36,719.67	1808.00	5,905.60	6,055.71	34.42	5,835.27	5,973.60	85.11	**5,804.19**	5,905.71	7.83

TABLE V
RESULTS OBTAINED FROM DIFFERENT HEURISTIC ALGORITHMS FOR THE INSTANCE SETS S_1 AND S_2 CONSIDERING 35 VEHICLES

No.	POPMUSIC			VNS(200000)			VNS(500000)			PLS		
	Best	Avg	Time(s)	Best	Avg	Time(s)	Best	Avg	Time(s)	Best	Avg	Time(s)
p01	1469.90	1852.09	1201.85	716.93	721.34	24.82	714.20	718.72	62.53	**713.18**	717.44	2.24
p02	2084.70	2625.83	2103.17	714.11	721.36	25.60	714.57	717.76	66.07	**713.59**	716.78	2.22
p03	3820.91	3955.16	1504.80	973.41	988.05	34.09	970.78	979.18	82.81	**952.17**	959.03	3.37
p04	6908.25	6908.25	604.90	1,989.72	2,012.44	44.66	1,989.17	1,999.85	107.66	**1,955.82**	1,959.14	5.22
p05	15394.49	15394.49	604.81	2,013.81	2,039.05	42.83	2,016.59	2,029.46	104.98	**1,985.03**	1,988.46	5.12
p06	6201.52	6201.52	906.36	1,594.01	1,638.75	48.00	1,596.34	1,617.49	122.44	**1,553.88**	1,563.22	4.57
p07	5882.11	5882.11	1207.36	1,555.21	1,597.65	48.86	1,561.25	1,580.43	122.37	**1,522.68**	1,528.43	5.30
p08	105289.18	105289.18	944.82	16,212.03	16,461.42	110.77	15,781.12	16,024.45	281.61	**15,410.92**	15,458.51	20.46
p09	78484.63	78484.63	953.47	13,974.04	14,370.73	122.54	13,547.48	13,813.17	308.73	**13,136.24**	13,335.06	18.60
p10	74570.66	74570.66	1260.00	13,051.13	13,444.28	126.42	12,437.61	12,898.75	305.83	**12,096.90**	12,432.72	18.64
p11	64744.95	64744.95	1566.57	13,159.95	13,582.67	118.01	12,519.78	12,829.25	283.86	**12,033.48**	12,228.64	17.34
p12	27202.35	27502.91	1243.83	**2,897.06**	2,897.31	36.04	**2,897.06**	2,897.06	89.72	**2,897.06**	2,897.06	2.02
p15	56249.30	56309.40	2435.71	5,854.11	6,231.24	65.68	**5,794.11**	5,918.35	173.39	**5,794.11**	5,794.11	5.20
p18	84494.16	84494.16	1867.21	12,655.91	13,137.37	105.23	11,843.29	12,145.91	266.37	**11,469.49**	11,546.91	12.33
pr01	4120.95	5380.99	2404.22	1,263.17	1,266.38	24.95	**1,261.77**	1,264.10	64.09	1,261.81	1,264.74	2.05
pr02	21969.60	24270.99	1206.95	2,608.14	2,632.31	43.25	2,596.20	2,619.00	108.56	**2,572.84**	2,580.94	4.82
pr03	40930.50	40930.50	1216.86	4,625.26	4,708.91	64.47	4,549.12	4,661.23	161.02	**4,466.10**	4,511.37	7.52
pr04	50580.46	50580.46	1230.68	6,165.38	6,353.07	83.14	6,015.15	6,175.39	221.38	**5,813.87**	5,863.56	12.77
pr05	50022.89	50022.89	1253.73	7,885.02	8,141.32	107.72	7,613.08	7,740.73	271.64	**7,157.06**	7,225.66	20.33
pr06	68366.90	68366.90	1277.04	10,051.62	10,356.10	129.01	9,435.73	9,628.95	320.63	**8,685.68**	8,874.53	27.29
pr07	9387.03	9996.68	2107.87	1,738.16	1,761.62	34.96	1,736.49	1,750.31	89.85	**1,727.25**	1,736.14	3.56
pr08	32020.00	32463.56	1821.83	4,142.28	4,307.24	66.06	4,115.09	4,197.41	164.16	**4,023.21**	4,046.68	8.20
pr09	42748.00	42748.00	1853.91	6,505.47	6,779.66	94.26	6,302.89	6,472.66	235.56	**5,937.19**	6,043.29	14.68
pr10	61794.38	61794.38	1902.27	10,768.12	11,376.31	131.95	9,983.57	10,396.66	320.59	**9,166.57**	9,336.73	28.60

solution for the MDCVRP model may not be the best one for the MDCCVRP model. For each testing instance, the best-known solution under the MDCVRP model is presented on the website: http://www.bernabe.dorronsoro.es/vrp/. The symbols used in Table II are in accordance with those in Table I. Given the best-known solution for MDCVRP, its cost under the MDCCVRP model is recalculated, as shown in the column F(MDCVRP) of Table II. Column F(MDCCVRP) means the best solution for the MDCCVRP obtained in our computational tests. To show the difference between the F(MDCVRP) and F(MDCCVRP) more clearly, GAP_1 is defined as follows:

$$\mathrm{GAP}_1 = \frac{\text{F(MDCCVRP)-F(MDCVRP)}}{\text{F(MDCVRP)}} \times 100\%.$$

From Table II, it is clear that the best-known solutions of the MDCVRP are not suitable for the MDCCVRP. Under the data sets S_1 and S_2, the average GAP_1 values are -26.09% and -35.56%, respectively.

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
RESULTS OBTAINED FROM DIFFERENT HEURISTIC ALGORITHMS FOR THE STANDARD MDCVRP INSTANCES

No.	GRASP	POPMUSIC		VNS(200000)		VNS(500000)		PLS	
	Best	Best	Avg	Best	Avg	Best	Avg	Best	Avg
p01	2363.12	1189.44	1215.09	1067.92	1116.97	1077.65	1109.34	**1055.35**	1095.44
p02	6835.93	3062.10	3133.57	2048.34	2166.43	**2043.13**	2137.62	2055.40	2149.54
p03	7616.12	1882.61	2124.76	1802.53	1875.11	1787.66	1846.87	**1758.70**	1809.51
p04	9692.48	3078.55	3283.10	2677.61	2796.33	**2618.44**	2722.03	2618.88	2703.37
p05	17803.39	8031.62	8611.04	3871.84	4020.70	3852.55	3943.83	**3766.20**	3824.49
p06	9839.65	2519.28	2595.44	2168.84	2265.49	2174.44	2236.55	**2160.93**	2188.17
p07	9417.83	2424.89	2431.15	2210.89	2286.16	2174.60	2226.93	**2142.11**	2181.18
p08	115018.18	37901.78	43338.98	19150.34	20042.37	18281.53	18907.79	**17393.46**	17862.13
p09	117090.49	31692.23	33034.68	16414.31	17194.11	15515.66	16735.37	**15041.69**	15448.98
p10	118584.70	30752.28	31073.05	15938.84	16564.25	14970.77	15523.86	**14265.77**	14619.35
p11	119292.87	31321.43	33328.83	15822.61	16590.89	14898.18	15473.21	**14381.43**	14552.90
p12	25911.71	9850.30	10388.31	5511.66	5641.23	**5494.36**	5562.86	**5494.36**	5536.40
p13	44077.56	6790.65	7479.01	4916.37	5016.86	**4914.66**	4962.44	**4914.66**	4926.54
p14	38793.96	5165.82	5238.74	4515.46	4600.37	**4510.12**	4542.56	**4510.12**	4512.28
p15	85752.11	23032.51	24149.00	11092.68	11513.79	10755.38	11128.82	**10662.27**	10747.26
p16	106550.37	22390.97	22514.86	10534.46	10848.58	10197.48	10389.14	**10086.50**	10122.13
p17	107998.43	17125.32	17663.09	9809.51	10243.77	9742.25	9939.57	**9538.69**	9573.86
p18	151889.84	36270.06	40761.55	17571.10	18690.62	16657.73	17207.98	**15912.27**	16072.75
p19	223941.44	34018.83	34027.64	16838.22	17772.34	15836.39	16395.69	**15255.02**	15370.98
p20	103530.09	30680.29	30713.07	15719.93	16976.94	15233.94	15752.56	**14709.23**	14786.87
p21	308607.93	63979.60	65202.24	35052.77	37346.66	28924.64	30173.10	**25770.35**	26102.29
p22	324751.44	60774.44	64574.36	32280.93	34313.75	27577.53	28529.03	**24451.01**	24816.85
p23	329474.92	62459.70	64047.74	30993.99	32951.28	26695.35	27617.71	**23656.13**	23925.16
pr01	15290.61	7225.27	7225.27	3748.11	3924.86	**3748.11**	3849.10	**3748.11**	3773.27
pr02	31196.70	9768.03	10706.88	5061.98	5376.33	5016.01	5229.21	**4973.36**	5000.74
pr03	76728.04	23599.30	24003.85	8625.79	9185.84	8476.45	8874.58	**8357.54**	8470.56
pr04	72858.39	30497.37	30664.53	10554.48	11382.38	9937.19	10597.57	**9274.00**	9585.47
pr05	81855.20	31833.45	32467.40	12187.41	12895.89	11014.60	11911.18	**10075.01**	10283.54
pr06	107770.70	33928.05	36592.41	12896.39	14106.53	12063.75	12522.24	**11071.00**	11234.57
pr07	25431.27	8739.62	8739.62	4941.39	5214.32	4917.63	5149.60	**4877.86**	4906.62
pr08	53183.55	15588.07	15966.36	7688.16	8211.18	7352.08	7716.02	**7141.88**	7265.17
pr09	92816.54	25381.49	25700.25	10735.73	11490.98	9927.48	10349.00	**9219.95**	9350.34
pr10	106172.13	33031.82	34105.06	14218.66	15347.61	12995.46	13713.06	**11693.45**	11811.29

C. Case Study

A case study for transporting medical supplies is presented in this section. Mete and Zabinsky [40] presented a case study for transporting medical supplies from warehouses to hospitals for disaster management. That case study [40] is based on the discussions with an Emergency Management Coordinator of a large Seattle medical center, and considers ten hospitals and five warehouses. The hospitals are numbered from 1 to 10, without giving their real names. Due to the lack of distances, information between hospitals, the case instance in [40] cannot be used directly in this article.

Suppose that the decision maker for the disaster management wants to minimize the total waiting time of each hospital for fast response and fairness. The MDCCVRP models this situation. Similar to [40], a case study is presented by considering 19 second-tier (and above) hospitals and three warehouses in Dalian, China. The three warehouses are located in: 1) Dalian Railway Station; 2) Dalian North Railway Station; and 3) Dalian Zhoushuizi International Airport. The distances are the real driving distances measured with Baidu map, and the demand amounts for each hospital are estimated using both population density and hospital capacity. The original data sets for the real-case study can be obtained upon request.

Table III shows the results for the real-case study. The same time limit (2 h) as that in [12] is set to the CPLEX optimizer for solving the MDCCVRP model. Apparently, when solving the real-case instance, the CPLEX optimizer is still searching for a better result after 2 h. The cost of the obtained best solution within the time limit is 151006.00, as shown in Table III. For the case study instance, the costs of the best solutions obtained by POPMUSIC and our PLS are 150642.00 and 148794.00, respectively. The computational times of POPMUSIC and our PLS are 110.16 s and 1.13 s, respectively. Therefore, our PLS yields a better solution with less computational time.

D. Comparisons Among Different Heuristic Methods

This section compares different methods in the aspects of solution quality and computational time. Lalla-Ruiz and Voß [12] provided 18 small and medium instances (i.e., S_3) for testing the MDCCVRP. In Table IV, the best values are shown in boldface. The results for POPMUSIC were copied from the published paper [12]. As shown in Table IV, columns 2 to 4 show the solutions obtained by POPMUSIC, and columns 5 to 7 show the solutions obtained by VNS with 200 000 iterations, and columns 8 to 10 show the solutions obtained by VNS with 500 000 iterations, and columns 11 to 13 show the solutions obtained by our PLS algorithm. The cost of the obtained

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
COMPUTATIONAL TIME (IN SECONDS) FOR POPMUSIC, VNS, AND PLS ALGORITHMS

No.	POPMUSIC	VNS(200000)	VNS(500000)	PLS
p01	13183.30	9.80	24.52	1.46
p02	3150.28	12.51	31.44	3.17
p03	19501.47	12.31	29.67	3.26
p04	27002.45	23.78	58.25	8.38
p05	4950.53	14.81	38.11	9.94
p06	31510.12	16.46	41.50	5.18
p07	57396.95	15.18	38.31	5.98
p08	53435.03	59.07	158.21	35.20
p09	66955.04	59.48	157.09	26.93
p10	57936.75	49.32	131.59	28.55
p11	53761.20	45.43	138.36	28.35
p12	9901.26	11.04	28.06	3.90
p13	13501.06	9.17	23.56	4.05
p14	23854.07	8.50	21.47	3.88
p15	20136.44	23.11	56.67	12.59
p16	15166.57	20.26	49.87	10.71
p17	27940.58	18.80	46.29	10.60
p18	33318.82	38.64	95.45	20.56
p19	25054.18	35.11	84.48	18.95
p20	66140.14	31.76	77.91	18.24
p21	59017.70	97.05	253.19	39.62
p22	76089.88	78.33	207.76	36.86
p23	48029.27	68.34	181.64	32.68
AVG	35084.05	32.97	85.80	16.05
pr01	1200.32	6.52	16.00	3.37
pr02	5250.63	11.66	28.85	9.28
pr03	9300.87	17.96	44.39	18.57
pr04	13352.94	31.57	77.97	39.79
pr05	13955.40	50.73	150.92	52.86
pr06	34843.14	39.92	99.38	57.00
pr07	1800.25	8.79	21.68	6.58
pr08	5850.51	18.00	44.28	18.75
pr09	17408.92	28.00	74.20	36.93
pr10	14552.92	43.53	115.05	54.05
AVG	25493.12	32.67	84.40	23.73

TABLE VIII
STABILITY COMPARISONS BETWEEN VNS AND PLS ALGORITHMS

No.	VNS(200000)		VNS(500000)		PLS	
	STDEV	CoV	STDEV	CoV	STDEV	CoV
p01	33.46	3.00%	29.51	2.66%	24.84	2.27%
p02	64.55	2.98%	54.09	2.53%	58.95	2.74%
p03	38.36	2.05%	28.83	1.56%	29.13	1.61%
p04	64.41	2.30%	49.53	1.82%	49.69	1.84%
p05	71.88	1.79%	62.71	1.59%	44.50	1.16%
p06	43.95	1.94%	40.31	1.80%	19.95	0.91%
p07	48.53	2.12%	27.96	1.26%	16.30	0.75%
p08	431.56	2.15%	262.32	1.39%	152.47	0.85%
p09	581.03	3.38%	677.18	4.05%	289.10	1.87%
p10	389.18	2.35%	270.86	1.74%	193.27	1.32%
p11	386.66	2.33%	262.90	1.70%	130.79	0.90%
p12	106.17	1.88%	71.38	1.28%	34.74	0.63%
p13	83.67	1.67%	43.10	0.87%	12.41	0.25%
p14	70.66	1.54%	44.57	0.98%	3.71	0.08%
p15	253.09	2.20%	230.77	2.07%	57.49	0.53%
p16	175.21	1.62%	143.28	1.38%	21.55	0.21%
p17	178.09	1.74%	127.88	1.29%	22.51	0.24%
p18	516.23	2.76%	360.66	2.10%	91.40	0.57%
p19	524.82	2.95%	284.81	1.74%	62.50	0.41%
p20	336.57	1.98%	197.47	1.25%	46.01	0.31%
p21	1172.14	3.14%	801.94	2.66%	193.88	0.74%
p22	1243.25	3.62%	686.18	2.41%	145.98	0.59%
p23	1113.71	3.38%	485.27	1.76%	120.12	0.50%
AVG	344.66	2.39%	227.98	1.82%	79.19	0.93%
pr01	110.33	2.81%	103.99	2.70%	36.52	0.97%
pr02	173.58	3.23%	135.09	2.58%	19.33	0.39%
pr03	294.80	3.21%	205.05	2.31%	56.24	0.66%
pr04	390.50	3.43%	319.83	3.02%	162.30	1.69%
pr05	394.42	3.06%	364.16	3.06%	106.90	1.04%
pr06	475.94	3.37%	227.68	1.82%	80.82	0.72%
pr07	146.21	2.80%	136.74	2.66%	22.53	0.46%
pr08	317.47	3.87%	189.91	2.46%	52.52	0.72%
pr09	416.46	3.62%	292.29	2.82%	54.55	0.58%
pr10	463.65	3.02%	345.40	2.52%	71.97	0.61%
AVG	318.34	3.24%	232.01	2.59%	66.37	0.78%

best solution (Best) and the average cost of the solutions in different runs (Avg) are shown in the table. The computational times are reported in seconds. POPMUSIC always decomposes a problem into several subproblems and uses exact methods to solve the subproblems. Therefore, POPMUSIC always requires a lot of computational time. As shown in Table IV, VNS and our PLS perform better than POPMUSIC with less computational time. Compared to VNS, our PLS performs better for all instances except for two instances (lr11 and lr12). For all instances, our PLS requires the least computational time.

In [12], POPMUSIC was also verified on the slightly modified MDCVRP instances with the number of vehicles fixed to 35. In Table V, the results for POPMUSIC were also copied from the published paper [12]. Similarly, the two local search-based methods (VNS and PLS) perform better than POPMUSIC with much less computational time. It is important to note that our PLS always performs best with the least computational time for all testing instances in Table V.

In this section, different heuristics are also tested on the standard MDCVRP instances. Detailed information on the standard MDCVRP instances is shown in Table II. Table VI compares four different heuristics in the aspects of solution quality, and Table VII compares different heuristics in the aspects of computational time. The GRASP method is a constructive heuristic method with no iterative improvement on solutions. For all the testing instances, GRASP always yields the final solution within 1 s. The computational time of GRASP is not reported in Table VII since it is very small. As shown in Table VI, our PLS algorithm performs best for 31 (out of 33) instances. For instances p02 and p04, the VNS method performs slightly better than our PLS algorithm. For the large scale problems ($|C| > 200$), the advantage of our PLS is more obvious. Take the instance P21, as an example, the best solution obtained by VNS with 200 000 iterations, VNS with 500 000 iterations, and PLS are 35052.77, 28924.64, and 25770.35, respectively. The GAPs are 26.48% and 10.91%. In Table VII, POPMUSIC always requires the longest computational time. For most testing instances, our PLS algorithm requires less time than VNS with 200 000 iterations. For all testing instances, our PLS algorithm requires less time than VNS with 500 000 iterations. In total, compared to other methods (POPMUSIC and VNS), our PLS method requires less computational time while providing better solutions.

E. Stability Analysis

In order to verify the stability of the algorithm, we introduce the coefficient of variation (CoV) and the standard

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
RESULTS OF THE TWO-SIDED WILCOXON RANK SUM TEST

No.	p01	p02	p03	p04	p05	p06	p07	p08	p09	p10	p11
TEST1	4.34E-03	1.45E-01	2.55E-08	1.19E-07	2.75E-11	1.46E-09	1.84E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11
TEST2	3.23E-02	2.10E-01	6.25E-06	5.77E-02	1.75E-09	1.76E-07	4.24E-09	1.51E-11	3.69E-10	2.04E-11	1.51E-11
No.	p12	p13	p14	p15	p16	p17	p18	p19	p20	p21	p22
TEST1	6.54E-05	1.34E-08	3.78E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11
TEST2	3.39E-01	4.87E-06	2.34E-05	5.47E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11
No.	p23	pr01	pr02	pr03	pr04	pr05	pr06	pr07	pr08	pr09	pr10
TEST1	1.51E-11	2.35E-08	1.49E-11	1.51E-11	1.51E-11	1.51E-11	1.51E-11	1.64E-11	1.51E-11	1.51E-11	1.51E-11
TEST2	1.51E-11	9.39E-05	3.00E-11	6.64E-11	1.67E-11	1.51E-11	1.51E-11	7.93E-11	1.67E-11	1.51E-11	1.51E-11

deviation (STDEV), which are important indicators to measure the stability of a heuristic [31]. The definition of CoV is as follows:

$$\mathrm{CoV} = \frac{\sigma}{\mu} \times 100\%,$$

$$\text{where } \mu = \frac{1}{T}\sum_{i=1}^{T} \mathrm{cost}(s_i),$$

$$\sigma = \sqrt{\frac{1}{T}\sum_{i=1}^{T}(\mathrm{cost}(s_i) - \mu)^2}.$$

where T (set to be 30 here) refers to the number of runs, and s_i refers to the cost of the obtained best solution in the ith run. The smaller the CoV, the more stable the corresponding algorithm is. As shown in Table VIII, for each testing instance, the CoV value of PLS algorithm is smaller than that of VNS. For the instances S_1, the average CoV for VNS(200 000), VNS(500 000), and PLS algorithms are 2.39%, 1.82%, and 0.93%, respectively. For the instances S_2, the average CoV for VNS(200 000), VNS(500 000), and PLS algorithms are 3.24%, 2.59%, and 0.78%, respectively. To sum up, our proposed PLS is more stable than the VNS method.

F. Statistical Analysis

Tables IV–VII clearly show that our proposed method performs better than the other two methods, and Table VIII shows that our method is the most stable one among the three methods. The VNS and our PLS perform much better than POPMUSIC. To show the significance of the performance difference between VNS and our PLS, we conduct the two-sided Wilcoxon rank sum test for hypothesis testing (TEST1) between our PLS method and VNS(200 000), and the two-sided Wilcoxon rank sum test for hypothesis testing (TEST2) between our PLS method and VNS(500 000). Table IX shows the p-value under each testing instance. The p-values of TEST1 are less than 0.005 for all instances except for the instance p02. The p-values of TEST2 are less than 0.005 for all instances except for four instances p01, p02, p04, and p12. Tables VI and IX together imply that our proposed PLS method performs statistically significantly better than the other methods for 29 testing instances. For four testing instances, the VNS (500 000) and our PLS are comparable in performance.

VI. CONCLUSION

The MDCCVRP is an extension of the CCVRP. The MDCCVRP is widely used in the distribution of disaster relief materials as well as in the fresh food transportation. In these applications, the arrival times at the customers are important, and good routing methodologies must be developed. In this article, the mathematical model for the MDCCVRP is presented, and a highly efficient PLS is developed for solving the problem. On the real case and standard benchmarking instances, the PLS algorithm outperforms the published heuristics in the aspects of solution quality, computational time, as well as stability.

More work is still needed to further generalize this article to cope with real-word challenges in disaster logistics and vital goods transportation. One promising extension is the MDCCVRP with split deliveries. Another extension is the multitrip version of the MDCCVRP. These two extensions are challenging while they can lead to a better use of vehicles. These two problems will be investigated in our future work.

ACKNOWLEDGMENT

The authors would like to thank the corresponding author of paper [39] (Y. Kuo) for kindly providing their algorithm codes and answering several questions. They would also like to thank the corresponding author of paper [12] (E. Lalla-Ruiz) for answering some technical questions and also thank the Editor Prof. Chen, the Associate Editor Prof. Tiwari, and the four anonymous reviewers for their constructive and important comments which led to major improvements of this article.

REFERENCES

[1] K. Braekers, K. Ramaekers, and I. V. Nieuwenhuyse, "The vehicle routing problem: State of the art classification and review," *Comput. Ind. Eng.*, vol. 99, pp. 300–313, Sep. 2016.
[2] X. Wang, T.-M. Choi, H. Liu, and X. Yue, "Novel ant colony optimization methods for simplifying solution construction in vehicle routing problems," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 11, pp. 3132–3141, Nov. 2016.
[3] K. Dorling, J. Heinrichs, G. G. Messier, and S. Magierowski, "Vehicle routing problems for drone delivery," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 47, no. 1, pp. 70–85, Jan. 2017.
[4] S. U. Ngueveu, C. Prins, and R. W. Calvo, "An effective memetic algorithm for the cumulative capacitated vehicle routing problem," *Comput. Oper. Res.*, vol. 37, no. 11, pp. 1877–1885, 2010.
[5] A. M. Campbell, D. Vandenbussche, and W. Hermann, "Routing for relief efforts," *Transport. Sci.*, vol. 42, no. 2, pp. 127–145, 2008.
[6] M. Vanajakumari, S. Kumar, and S. Gupta, "An integrated logistic model for predictable disasters," *Product. Oper. Manag.*, vol. 25, no. 5, pp. 791–811, 2016.
[7] L. Zhen, K. Wang, and H.-C. Liu, "Disaster relief facility network design in metropolises," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 45, no. 5, pp. 751–761, May 2015.
[8] J. Lysgaard and S. Wøhlk, "A branch-and-cut-and-price algorithm for the cumulative capacitated vehicle routing problem," *Networks*, vol. 236, no. 3, pp. 800–810, 2014.

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.

[9] S. Nucamendi-Guillen, F. Angel-Bello, I. Martínez-Salazar, and A. Cordero-Franco, "The cumulative capacitated vehicle routing problem: New formulations and iterated greedy algorithms," *Expert Syst. Appl.*, vol. 113, pp. 315–327, Dec. 2018.

[10] J. Du, L. Xiang, L. Yu, R. Dan, and J. Zhou, "Multi-depot vehicle routing problem for hazardous materials transportation: A fuzzy bilevel programming," *Inf. Sci.*, vol. 399, pp. 201–218, Aug. 2017.

[11] J. Wang, T. Weng, and Q. Zhang, "A two-stage multiobjective evolutionary algorithm for multiobjective multidepot vehicle routing problem with time windows," *IEEE Trans. Cybern.*, vol. 49, no. 7, pp. 2467–2478, Jul. 2019.

[12] E. Lalla-Ruiz and S. Voß, "A POPMUSIC approach for the multi-depot cumulative capacitated vehicle routing problem," *Optim. Lett.*, to be published.

[13] P. Shaw, "A new local search algorithm providing high quality solutions to vehicle routing problems," Univ. Strathclyde, Glasgow, U.K., Rep., 1997.

[14] J. Brandão, "Iterated local search algorithm with ejection chains for the open vehicle routing problem with time windows," *Comput. Ind. Eng.*, vol. 120, pp. 146–159, Jun. 2018.

[15] E. E. Zachariadis and C. T. Kiranoudis, "An effective local search approach for the vehicle routing problem with backhauls," *Expert Syst. Appl.*, vol. 39, no. 3, pp. 3174–3184, 2012.

[16] M. E. Mcnabb, J. D. Weir, R. R. Hill, and S. N. Hall, "Testing local search move operators on the vehicle routing problem with split deliveries and time windows," *Comput. Oper. Res.*, vol. 56, pp. 93–109, Apr. 2015.

[17] G. M. Ribeiro and G. Laporte, "An adaptive large neighborhood search heuristic for the cumulative capacitated vehicle routing problem," *Comput. Oper. Res.*, vol. 39, no. 3, pp. 728–735, 2012.

[18] N. Smiti, M. M. Dhiaf, B. Jarboui, and S. Hanafi, "Skewed general variable neighborhood search for the cumulative capacitated vehicle routing problem," *Int. Trans. Oper. Res.*, vol. 27, no. 1, pp. 651–664, Feb. 2018.

[19] L. Ke and Z. Feng, "A two-phase metaheuristic for the cumulative capacitated vehicle routing problem," *Comput. Oper. Res.*, vol. 40, no. 2, pp. 633–638, 2013.

[20] L. Ke, "A brain storm optimization approach for the cumulative capacitated vehicle routing problem," *Memetic Comput.*, vol. 10, no. 4, pp. 411–421, 2018.

[21] R. Liu and Z. Jiang, "A hybrid large-neighborhood search algorithm for the cumulative capacitated vehicle routing problem with time-window constraints," *Appl. Soft Comput.*, vol. 80, pp. 18–30, Jul. 2019.

[22] J. C. Rivera, H. M. Afsar, and C. Prins, "A multistart iterated local search for the multitrip cumulative capacitated vehicle routing problem," *Comput. Optim. Appl.*, vol. 61, no. 1, pp. 159–187, 2015.

[23] J. C. Rivera, H. M. Afsar, and C. Prins, "Mathematical formulations and exact algorithm for the multitrip cumulative capacitated single-vehicle routing problem," *Eur. J. Oper. Res.*, vol. 249, no. 1, pp. 93–104, 2016.

[24] I. Martinez-Salazar, F. Angel-Bello, and A. Alvarez, "A customer-centric routing problem with multiple trips of a single vehicle," *J. Oper. Res. Soc.*, vol. 66, no. 8, pp. 1312–1323, 2015.

[25] J. Fakcharoenphol, C. Harrelson, and S. Rao, "The k-traveling repairmen problem," *ACM Trans. Algorithms*, vol. 3, no. 4, p. 40, 2007.

[26] S. Nucamendi-Guillen, I. Martinez-Salazar, F. Angel-Bello, and J. M. Moreno-Vega, "A mixed integer formulation and an efficient metaheuristic procedure for the k-travelling repairmen problem," *J. Oper. Res. Soc.*, vol. 67, no. 8, pp. 1121–1134, 2016.

[27] M. M. Silva, A. Subramanian, T. Vidal, and L. S. Ochi, "A simple and effective metaheuristic for the minimum latency problem," *Eur. J. Oper. Res.*, vol. 221, no. 3, pp. 513–520, 2012.

[28] T. Bulhoes, R. Sadykov, and E. Uchoa, "A branch-and-price algorithm for the minimum latency problem," *Comput. Oper. Res.*, vol. 93, pp. 66–78, May 2018.

[29] I. Kara, B. Kara, and M. K. Yetis, *Cumulative Vehicle Routing Problems*, T. Caric and H. Gold, Eds. Vienna, Austria: InTech Educ., 2008.

[30] J. Tang, J. Guan, Y. Yu, and J. Chen, "Beam search combined with MAX–MIN ant systems and benchmarking data tests for weighted vehicle routing problem," *IEEE Trans. Autom. Sci. Eng.*, vol. 11, no. 4, pp. 1097–1109, Oct. 2014.

[31] X. Wang, T.-M. Choi, H. Liu, and X. Yue, "A novel hybrid ant colony optimization algorithm for emergency transportation problems during post-disaster scenarios," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 48, no. 4, pp. 545–556, Apr. 2018.

[32] Q. Hui and H. Zhang, "Optimal balanced coordinated network resource allocation using swarm optimization," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 45, no. 5, pp. 770–787, May 2015.

[33] K. Wang, H. Luo, F. Liu, and X. Yue, "Permutation flow shop scheduling with batch delivery to multiple customers in supply chains," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 48, no. 10, pp. 1826–1837, Oct. 2018.

[34] Y.-H. Jia *et al.*, "A dynamic logistic dispatching system with set-based particle swarm optimization," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 48, no. 9, pp. 1607–1621, Sep. 2018.

[35] L. Kang, J. Wu, H. Sun, X. Zhu, and Z. Gao, "A case study on the coordination of last trains for the Beijing subway network," *Transport. Res. B Methodol.*, vol. 72, no. 72, pp. 112–127, 2015.

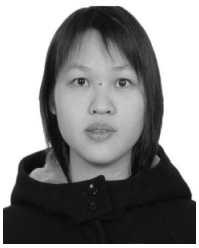
[36] O. Bräysy and M. Gendreau, "Vehicle routing problem with time windows, part I: Route construction and local search algorithms," *Transport. Sci.*, vol. 39, no. 1, pp. 104–118, 2005.

[37] F. Li, B. Golden, and E. Wasil, "Very large-scale vehicle routing: New test problems, algorithms, and results," *Comput. Oper. Res.*, vol. 32, no. 5, pp. 1165–1179, 2005.

[38] P. Schittekat, J. Kinable, K. Sörensen, M. Sevaux, F. Spieksma, and J. Springael, "A metaheuristic for the school bus routing problem with bus stop selection," *Eur. J. Oper. Res.*, vol. 229, no. 2, pp. 518–528, 2013.

[39] Y. Kuo and C.-C. Wang, "A variable neighborhood search for the multi-depot vehicle routing problem with loading cost," *Expert Syst. Appl.*, vol. 39, no. 8, pp. 6949–6954, 2012.

[40] H. O. Mete and Z. B. Zabinsky, "Stochastic optimization of medical supply location and distribution in disaster management," *Int. J. Prod. Econ.*, vol. 126, no. 1, pp. 76–84, 2010.



Xinyu Wang received the Ph.D. degree in computer science and engineering from Tsinghua University, Beijing, China, in 2008.

She is currently an Associate Professor with the Department of Management Science and Engineering, Dongbei University of Finance and Economics, Dalian, China. Her current research interests include information processing, decision-making, and operational management.

Tsan-Ming Choi received the Ph.D. degree in systems engineering and engineering management from The Chinese University of Hong Kong, Hong Kong, in 2002.

He is currently a Full Professor with The Hong Kong Polytechnic University, Hong Kong. He has authored/edited 16 books and published extensively in Web of Science indexed journals, such as *Automatica*, *Decision Sciences*, *European Journal of Operational Research*, IEEE TRANSACTIONS ON AUTOMATIC CONTROL, IEEE TRANSACTIONS AUTOMATION SCIENCE AND ENGINEERING, IEEE TRANSACTIONS ON CYBERNETICS, IEEE TRANSACTIONS ON ENGINEERING MANAGEMENT, IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS, *Journal of Operations Management*, *Naval Research Logistics*, *Production and Operations Management*, *Service Sciences*, and *Transportation Research—Part E: Logistics and Transportation Review*. His current research interests include supply chain and logistics management and applied optimization.

Prof. Choi is the Co-Editor-in-Chief of *Transportation Research—Part E: Logistics and Transportation Review*, a Senior Editor of *Production and Operations Management*, and an Associate Editor of IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS and *Information Sciences*.

Zhiying Li received the bachelor's degree from the Department of Management Science and Engineering, Dongbei University of Finance and Economics, Dalian, China, in 2019.

Her current research interest includes intelligent optimization.

Shuai Shao received the bachelor's degree from the Department of Economics and Management, Shandong University of Science and Technology, Qingdao, China, in 2017.

His current research interests include operations management and optimization.

Authorized licensed use limited to: Indian Institute of Information technology Sricity. Downloaded on March 13,2025 at 13:42:22 UTC from IEEE Xplore. Restrictions apply.